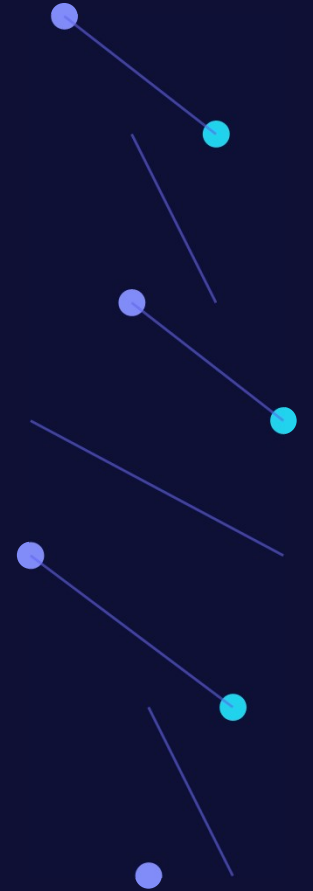




Fundamentos de IA

De cómo funciona un modelo de lenguaje por dentro, a cómo comunicarte con él de forma experta.





Historia y Transformers

De la IA simbólica al mecanismo de atención



1.1 · HISTORIA

70 años en una línea de tiempo

Test de Turing

Alan Turing pregunta:
¿puede pensar una
máquina?

1950

Nace el término

Conferencia de Dartmouth:
se acuña «Inteligencia
Artificial».

1956

Deep Blue

La IA vence al campeón
mundial de ajedrez
Kasparov.

1997

Deep Learning

AlexNet dispara la
revolución de las redes
neuronales.

2012

Transformers

«Attention is All You Need»:
nace la arquitectura de los
LLM.

2017

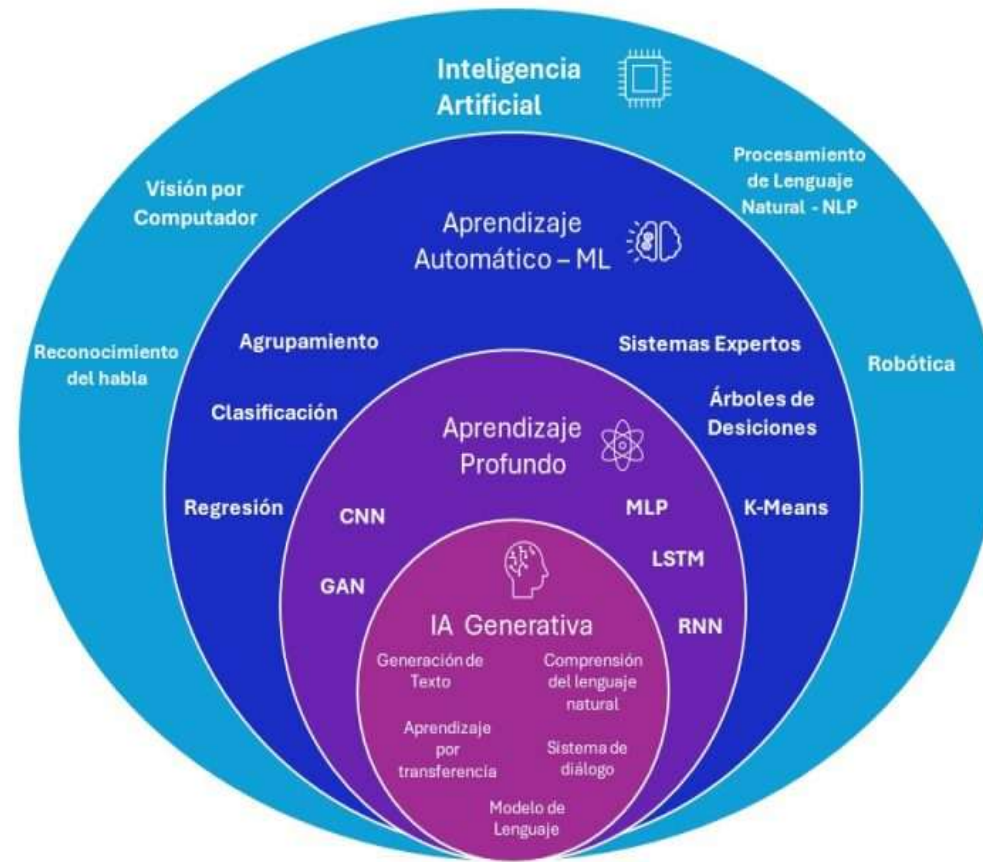
Era de los LLM

ChatGPT, Claude y compañía
llegan al público masivo.

2022+



1.1 · ENFOQUES





1.1 · ENFOQUES

Tres formas de «hacer» IA



IA Simbólica

Reglas escritas a mano

El humano programa cada regla: «Si fiebre Y tos ENTONCES...». Rígida y no escala.

Basada en lógica y reglas explícitas.



Machine Learning

Aprender de los datos

En lugar de reglas, el modelo detecta patrones en ejemplos. Necesita datos etiquetados.

Aprende patrones estadísticos.



Deep Learning / LLM

Redes neuronales profundas

Muchas capas que aprenden representaciones cada vez más abstractas. Base de los LLM.

Aprende representaciones jerárquicas.



1.2 · 2017

«Attention is All You Need»

El paper que lo cambió todo

Investigadores de Google proponen una arquitectura que **elimina la recurrencia** por completo. En su lugar, usa solo **atención** para relacionar todas las palabras entre sí, en paralelo.



Paralelización — entrenamiento mucho más rápido en GPU



Contexto largo — relaciona palabras distantes sin perder señal



Escala — más datos + más parámetros = mejor

Arquitectura (simplificada)





1.2 · MECANISMO DE ATENCIÓN

¿Qué es «prestar atención»?

Para entender una palabra, el modelo mira las demás y decide cuáles son relevantes. Ejemplo:

El **perro** no alcanzó la pelota porque **era** muy lento

Palabra analizada

La pregunta implícita

«**era**» ¿se refiere a qué? ¿al **perro** o a la pelota? La atención asigna **más peso a «perro»** porque es quien puede ser «lento».

La idea en una frase

Cada palabra «**mira**» a todas las demás al mismo tiempo y **pondera** cuánto influye cada una en su significado. Ese peso es lo que aprende el modelo.



SESIÓN 2

LLMs por dentro + Prompting

Tokens, embeddings, probabilidad y la anatomía de un buen prompt



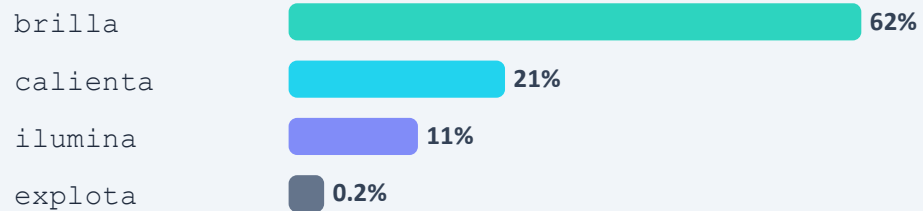
1.3 · DEFINICIÓN

¿Qué es, en esencia, un LLM?

Un modelo de lenguaje es, en el fondo, **una máquina de autocompletar extremadamente sofisticada**: dado un texto, predice cuál es el siguiente fragmento más probable.

Predicción del siguiente token

Entrada: «El cielo está despejado y el sol...»



Consecuencia clave

No «sabe» ni «entiende» como un humano. Estima probabilidades a partir de patrones. Por eso puede sonar seguro y equivocarse: predice lo *plausible*, no necesariamente lo *verdadero*.



1.3 · TOKENIZACIÓN

El modelo no lee letras: lee tokens

Un token es una pieza de texto (una palabra, parte de una palabra, un signo). El texto se parte en tokens antes de entrar al modelo.

«Inteligencia artificial es fascinante» →



6 tokens · «·» marca un espacio inicial



Se paga por token

El costo de la API se mide en tokens de entrada y salida, no en palabras.



Ventana de contexto

Cada modelo admite un máximo de tokens. Superarlo = «olvida» el inicio.



No todos igual

El español y el código suelen gastar más tokens que el inglés.

[OpenAI Tokenizer](#)



1.3 · EMBEDDINGS

Del token al vector: los embeddings

¿Qué es un embedding?

Cada token se convierte en una **lista de números (un vector)** que captura su significado. Palabras con sentido parecido tienen vectores **cercanos** en el espacio.

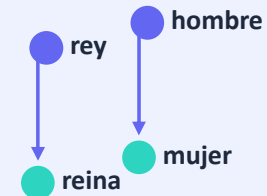
Ejemplo (simplificado a 4 dimensiones)

"**rey**" → [0.82, 0.15, 0.91, 0.44]

"**reina**" → [0.80, 0.63, 0.89, 0.41]

Casi idénticos salvo la dimensión «género».

El espacio semántico



gato
perro

La misma «dirección» separa rey→reina y hombre→mujer: el espacio codifica relaciones.



1.3 · SALIDA PROBABILÍSTICA

Temperatura: el dial de creatividad

El modelo produce una distribución de probabilidad sobre el vocabulario. La temperatura decide cuánto arriesga al elegir.



Temperatura baja (≈ 0)

Determinista y conservador

Elige casi siempre la opción más probable. Respuestas consistentes y predecibles.



Úsala para: Datos, código, clasificación, extracción.



Temperatura alta (≈ 1)

Diverso y creativo

Da más chance a opciones menos probables. Respuestas variadas y sorprendentes.



Úsala para: Lluvia de ideas, redacción creativa, marketing.



1.4 · INGENIERÍA DE PROMPT

El prompt es tu interfaz de programación

Con un LLM no programas con código, sino con **lenguaje natural**. La calidad de la respuesta depende, en gran medida, de la **calidad de la instrucción**.

Hace referencia al proceso de creación de una **instrucción** para conseguir que el modelo genere el **output deseado**.



Precisión

Un prompt claro reduce ambigüedad y respuestas genéricas.



Reproducibilidad

Buenas instrucciones dan resultados estables y confiables.



Eficiencia

Menos reintentos = menos tokens = menos costo y tiempo.



1.4 · ANATOMÍA

Un prompt efectivo: 4 ingredientes



Rol

Quién debe «ser» el modelo

«Actúa como un auditor de facturación con 10 años de experiencia.»



Tarea

Qué acción concreta pides

«Revisa estas lecturas y detecta consumos anómalos.»



Contexto

Datos y restricciones relevantes

«Umbral de alerta: variación > 40% respecto al promedio del cliente.»



Formato

Cómo quieres la salida

«Devuelve una tabla con: id, consumo, % variación, veredicto.»



1.4 · COMPARACIÓN

Prompt pobre vs. prompt estructurado



Vago

«Analiza estos datos de consumo.»

Resultado:

- ✗ Genérico e impredecible
- ✗ No sabe qué es «anómalo»
- ✗ Formato de salida al azar



Estructurado (R-T-C-F)

Actúa como analista de facturación. Revisa el consumo mensual y marca clientes con variación > 40% vs su promedio. Ignora meses con lectura estimada. Devuelve una tabla: id, consumo, %variación, veredicto.

Rol · **Tarea** · **Contexto** · **Formato**

Salida útil, consistente y lista para usar.



1.4 · ROLES DEL MENSAJE

System Prompt vs. User Prompt



System Prompt

Las reglas del juego

Define identidad, tono, límites y comportamiento GLOBAL. Se fija una vez y persiste en toda la conversación.

«Eres un asistente de EPM. Responde en español, formal y conciso. Nunca inventes datos de clientes.»



User Prompt

La petición puntual

Es cada mensaje concreto del usuario dentro de esas reglas. Cambia en cada turno.

«Resume el historial de facturación del contrato 10432 del último trimestre.»

Regla práctica: lo que vale para SIEMPRE va en el system; lo puntual, en el user.



SESIÓN 3

Técnicas avanzadas de prompting

Cómo hacer que el modelo razone, se corrija y actúe



1.4.3 · PANORAMA

Seis técnicas, de menor a mayor potencia



Zero-Shot

Pedir sin ejemplos

1



Few-Shot

Enseñar con ejemplos

2



Chain of Thought

Pensar paso a paso

3



Least-to-Most

Descomponer en subproblemas

4



Self-Consistency

Votar entre varias respuestas

5



ReAct

Razonar + usar herramientas

6



1.4.3 · LA FAMILIA «SHOT»

Zero-Shot vs. Few-Shot



Zero-Shot

Pides la tarea directamente, sin ejemplos. Rápido; ideal para tareas simples o comunes.

```
Clasifica el sentimiento  
de esta reseña:  
"El servicio fue lentísimo."  
  
Sentimiento: ■
```



Few-Shot

Das 2-3 ejemplos resueltos para fijar el patrón y el formato exacto que quieres.

```
"Excelente atención" → POSITIVO  
"Nunca más vuelvo" → NEGATIVO  
"Cumplió lo justo" → NEUTRO  
  
"El servicio fue lentísimo" → ■
```



1.4.3 · RAZONAMIENTO

Chain of Thought: «piensa paso a paso»

La idea

Pedirle al modelo que **muestre su razonamiento** antes de responder. Al «pensar en voz alta», comete menos errores en tareas de lógica, matemáticas y multi-paso.



El disparador mágico

Basta con añadir: «Razona paso a paso antes de dar la respuesta final.»

Sin CoT vs con CoT

«Si 3 máquinas leen 3 medidores en 3 min, ¿cuánto tardan 100 máquinas en leer 100 medidores?»

Sin CoT

«100 minutos»



respuesta rápida y errónea

Con CoT

«Cada máquina tarda 3 min por medidor → 100 máquinas en paralelo = 3 min.»





1.4.3 · DESCOMPOSICIÓN

Least-to-Most: divide y vencerás

La idea

Un problema complejo se **parte en subproblemas más simples**, que se resuelven en orden. Cada respuesta **alimenta** la siguiente.

Diferencia con CoT: CoT razona dentro de una respuesta; Least-to-Most encadena varias, ideal cuando el problema es demasiado grande para un solo paso.

Ejemplo: informe de morosidad

- 1 Extraer los contratos con pagos vencidos
- 2 Calcular días de mora por contrato
- 3 Agrupar por rango de mora (0-30, 31-60...)
- 4 Redactar el resumen ejecutivo final



1.4.3 · ROBUSTEZ

Self-Consistency: sabiduría de la mayoría

La idea

En vez de una respuesta, se generan **varias en paralelo** (con temperatura alta) y se **vota la más frecuente**. Filtra los razonamientos con errores aislados.

Costo

Multiplica las llamadas (y el costo). Resérvala para respuestas críticas donde equivocarse sale caro.

Misma pregunta, 3 caminos, 1 voto

Camino A

= 42



Camino B

= 42



Camino C

= 37



Voto
mayoritario

42

2 de 3 caminos coinciden en 42 → esa gana. El error aislado (37) se descarta.



1.4.3 · ACCIÓN

ReAct: razonar y actuar en ciclo

La idea

El modelo alterna entre **pensar** y **usar herramientas** (buscar, calcular, consultar una API o base de datos) hasta resolver la tarea.



Por qué importa

Rompe el límite del conocimiento fijo del modelo: puede consultar datos frescos y actuar. Es la base de los agentes autónomos.

El ciclo Thought → Action → Observation



Thought (Pensar)

«Necesito el consumo del contrato 10432»



Action (Actuar)

`consultar_bd(contrato=10432)`



Observation (Observar)

→ 245 m³ este mes



Answer (Responder)

«El consumo es 245 m³, 38% sobre su media»



1.4.3 · GUÍA DE DECISIÓN

¿Qué técnica uso y cuándo?

Técnica	Úsala cuando...	Costo
Zero-Shot	La tarea es simple o muy común	Bajo
Few-Shot	Necesitas un formato o estilo específico	Bajo
Chain of Thought	Hay lógica, matemáticas o varios pasos	Medio
Least-to-Most	El problema es grande y se puede descomponer	Medio
Self-Consistency	La respuesta es crítica y no puede fallar	Alto
ReAct	Se necesitan datos externos o herramientas	Alto

Regla general: empieza simple (zero/few-shot). Sube de nivel solo si la tarea lo exige — cada escalón cuesta más tokens y latencia.



Introducción a RAG (Retrieved Augmented Generation)

Cómo darle al modelo memoria fresca y confiable



1.5 · EL PROBLEMA

Dos debilidades de un LLM «a secas»



Alucinaciones

Como predice lo plausible, a veces inventa datos, cifras o citas con total seguridad. Suenan convincentes aunque sean falsos.



«El contrato 10432 se firmó en marzo de 2019» → dato inventado.



Conocimiento congelado

Solo sabe lo que vio durante su entrenamiento. Ignora tus documentos internos y todo lo posterior a su fecha de corte.



«No conozco tu política de facturación 2025.»



1.5 · LA SOLUCIÓN

RAG: examen con libro abierto

RAG (Retrieval-Augmented Generation) = antes de responder, el sistema **busca** información relevante en tus documentos y se la **entrega al modelo** para que responda basándose en ella.



Sin RAG: examen de memoria

El modelo responde solo con lo que recuerda. Si no lo sabe... improvisa.



Con RAG: libro abierto

Consulta tus fuentes en el momento y responde citándolas. **Menos alucinaciones y siempre actualizado.**



1.5 · CÓMO FUNCIONA

El flujo de RAG en 4 pasos



1 · Pregunta

El usuario pregunta algo sobre tus datos.



2 · Recuperar

Se buscan los fragmentos más relevantes (por similitud de embeddings)*.



3 · Aumentar

Esos fragmentos se añaden al prompt como contexto.



4 · Generar

El modelo responde basándose en el contexto recuperado.

Recuerda: la búsqueda del paso 2 usa los embeddings que vimos en la sesión 2 — se recupera por «cercanía de significado», no por palabras exactas. Previamente esta información debe de estar vectorizada y almacenada en una base de datos vectorial como Chroma o Pinecone.



1.5 · CRITERIO

RAG no es la única opción



Prompt largo

Pegar el documento entero en el prompt.

Rápido para 1 documento pequeño. No escala: consume muchos tokens y choca con el límite de contexto.

Mejor para:

Datos pequeños y puntuales



RAG

Buscar y traer solo lo relevante.

Escala a millones de documentos, se actualiza al instante y cita fuentes. Ideal para bases de conocimiento.

Mejor para:

Conocimiento amplio y cambiante



Fine-tuning

Reentrenar el modelo con tus datos.

Enseña estilo o comportamiento, no hechos frescos. Costoso y hay que repetirlo cuando los datos cambian.

Mejor para:

Cambiar tono o formato, no datos